



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

ASSIGNMENT 7

Abhinav Khandelwal—1024150194

1.

Class **polygon** contains data members width and height and public method **set_value()** to assign values. Classes **Rectangle** and **Triangle** are inherited from polygon class. Both classes contain public method **calculate_area()** to calculate the area.

Use base class pointer to access derived class object and show the area calculated.

main.cpp	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 class Polygon { 5 protected: 6 int width, height; 7 public: 8 void set_value(int w, int h) { 9 width = w; 10 height = h; 11 } 12 virtual void calculate_area() = 0; 13 }; 14 15 class Rectangle : public Polygon { 16 public: 17 void calculate_area() { 18 cout << "Area of Rectangle: " << width * 19 height << endl; 20 } 21 }; 22 23 class Triangle : public Polygon { 24 public: 25 void calculate_area() { 26 cout << "Area of Triangle: " << (width * 27 height) / 2 << endl; 28 } 29 };</pre>	<pre>Area of Rectangle: 50 Area of Triangle: 25 === Code Execution Successful ===</pre>

```

28 int main() {
29     Polygon *p;
30     Rectangle r;
31     Triangle t;
32     p = &r;
33     p->set_value(10, 5);
34     p->calculate_area();
35     p = &t;
36     p->set_value(10, 5);
37     p->calculate_area();
38     return 0;}

```

2.

Write a program to create a class **shape** with functions **area** and **display** to find area and display the name of the shape and other essential components.

Create derived classes **circle**, **rectangle**, and **triangle** each having overridden functions **area** and **display**.

Display the area of all shapes.

main.cpp

Share

Run

```

1 #include <iostream>
2 using namespace std;
3
4 class Shape {
5 public:
6     virtual void area() = 0;
7     virtual void display() = 0;
8 };
9
10 class Circle : public Shape {
11     float r;
12 public:
13     Circle(float x) { r = x; }
14
15     void area() {
16         cout << "Area: " << 3.14 * r * r << endl;
17     }
18
19     void display() {
20         cout << "Shape: Circle" << endl;
21     }
22 };
23
24 class Rectangle : public Shape {
25     int l, b;
26 public:
27     Rectangle(int x, int y) { l = x; b = y; }
28
29     void area() {
30         cout << "Area: " << l * b << endl;
31     }

```

Output

Clear

```

Shape: Circle
Area: 78.5
Shape: Rectangle
Area: 24
Shape: Triangle
Area: 12

=== Code Execution Successful ===

```

```
32
33 ▾ void display() {
34     cout << "Shape: Rectangle" << endl;
35 }
36 };
37
38 ▾ class Triangle : public Shape {
39     int b, h;
40 public:
41     Triangle(int x, int y) { b = x; h = y; }
42
43 ▾ void area() {
44     cout << "Area: " << (b * h) / 2 << endl;
45 }
46
47 ▾ void display() {
48     cout << "Shape: Triangle" << endl;
49 }
50 };
51
52 ▾ int main() {
53     Shape *s;
54
55     Circle c(5);
56     Rectangle r(4,6);
57     Triangle t(3,8);
58
59     s = &c; s->display(); s->area();
60     s = &r; s->display(); s->area();
61     s = &t; s->display(); s->area();
62
63     return 0;
64 }
```

3.

Write a C++ program to compute area of:

- Right angle triangle
 - Equilateral triangle
 - Isosceles triangle
- using function overloading.

main.cpp	Output
<pre>1 #include <iostream> 2 #include <cmath> 3 using namespace std; 4 5 float area(float b, float h) { 6 return 0.5 * b * h; 7 } 8 9 float area(float side) { 10 return (sqrt(3) / 4) * side * side; 11 } 12 13 float area(float a, float b, float c) { 14 float s = (a + b + c) / 2; 15 return sqrt(s * (s - a) * (s - b) * (s - c)); 16 } 17 18 int main() { 19 cout << "Right Triangle Area: " << area(5,6) << endl; 20 cout << "Equilateral Triangle Area: " << area(4) << endl; 21 cout << "Isosceles Triangle Area: " << area(5,5,6) << endl; 22 23 return 0; 24 }</pre>	<pre>Right Triangle Area: 15 Equilateral Triangle Area: 6.9282 Isosceles Triangle Area: 12 === Code Execution Successful ===</pre>

4.

Write a program with **Student** as abstract class and create derived classes:

- Engineering
- Medicine
- Science

Create objects of derived classes and access them using an array of base class pointers.

```
main.cpp  [Icons]  Run  Output  Clear

1  #include <iostream>
2  using namespace std;
3
4  class Student {
5  public:
6      virtual void show() = 0;
7  };
8
9  class Engineering : public Student {
10 public:
11     void show() {
12         cout << "Engineering Student" << endl;
13     }
14 };
15 class Medicine : public Student {
16 public:
17     void show() {
18         cout << "Medicine Student" << endl;
19     }
20 };
21 class Science : public Student {
22 public:
23     void show() {
24         cout << "Science Student" << endl;
25     }
26 };
27
28 int main() {
29     Student* s[3];
30     Engineering e;
31     Medicine m;
32     Science sc;
33     s[0] = &e;
34     s[1] = &m;
35     s[2] = &sc;
36
37     for(int i = 0; i < 3; i++) {
38         s[i]->show();
39     }
40     return 0;
}
```

Engineering Student
Medicine Student
Science Student

=== Code Execution Successful ===

5.

Create a class **Time** with private variables (h, m, s).

Overload **+** operator to add two time objects.

```
int main(){
```

```
    Time t1(5,15,34), t2(9,53,58), t3;
```

```
    t3 = t1 + t2;
```

```
    t3.show();
```

```
}
```

```
1  #include <iostream>
2  using namespace std;
3
4  class Time {
5      int h, m, s;
6
7  public:
8      Time(int a=0, int b=0, int c=0) {
9          h = a;
10         m = b;
11         s = c;
12     }
13
14     Time operator + (Time t) {
15         Time temp;
16
17         temp.s = s + t.s;
18         temp.m = m + t.m + temp.s / 60;
19         temp.s = temp.s % 60;
20
21         temp.h = h + t.h + temp.m / 60;
22         temp.m = temp.m % 60;
23
24         return temp;
25     }
26
27     void show() {
28         cout << h << ":" << m << ":" << s << endl;
29     }
30 };
31
```

15:9:32

=== Code Execution Successful ===

```
32 int main() {
33     Time t1(5,15,34), t2(9,53,58), t3;
34
35     t3 = t1 + t2;
36     t3.show();
37
38     return 0;
39 }
```

6.

Define a class **STRING** and overload:

- **==** to compare two strings
- **+** for concatenation

```
main.cpp  [Icons]  Run  Output

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class STRING {
6      string str;
7
8  public:
9      STRING(string s="") {
10         str = s;
11     }
12
13     bool operator == (STRING s) {
14         return str == s.str;
15     }
16
17     STRING operator + (STRING s) {
18         return STRING(str + s.str);
19     }
20
21     void show() {
22         cout << str << endl;
23     }
24 };
25
26 int main() {
27     STRING s1("Hello "), s2("World"), s3;
28
29     s3 = s1 + s2;
30     s3.show();
31
32     if(s1 == s2)
33         cout << "Strings are Equal\n";
34     else
35         cout << "Strings are Not Equal\n";
36
37     return 0;
38 }
```

Output

```
Hello World
Strings are Not Equal

=== Code Execution Successful ===
```

7.

Write a C++ program to create a class `matrix` and overload `*` operator using friend function to multiply two matrices.

```
1  #include <iostream>
2  using namespace std;
3
4  class Matrix {
5      int a[2][2];
6
7  public:
8      void input() {
9          for(int i=0;i<2;i++)
10             for(int j=0;j<2;j++)
11                 cin >> a[i][j];
12     }
13
14     void display() {
15         for(int i=0;i<2;i++) {
16             for(int j=0;j<2;j++)
17                 cout << a[i][j] << " ";
18             cout << endl;
19         }
20     }
21
22     friend Matrix operator*(Matrix, Matrix);
23 };
24
25 Matrix operator*(Matrix m1, Matrix m2) {
26     Matrix temp;
27
28     for(int i=0;i<2;i++)
29         for(int j=0;j<2;j++) {
30             temp.a[i][j] = 0;
31             for(int k=0;k<2;k++)
32                 temp.a[i][j] += m1.a[i][k] * m2
33                     .a[k][j];
34
35     return temp;
36 }
37
38 int main() {
39     Matrix m1, m2, m3;
40
41     cout << "Enter Matrix 1:\n";
42     m1.input();
43
44     cout << "Enter Matrix 2:\n";
45     m2.input();
46
47     m3 = m1 * m2;
48
49     cout << "Result:\n";
50     m3.display();
51
52     return 0;
53 }
```

Enter Matrix 1:
1 2 3 4
Enter Matrix 2:
5 6 7 8
Result:
19 22
43 50

=== Code Execution Successful ===

8.

Overload `[]` to check array index out of bounds at runtime.

main.cpp	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 class Array { 5 int a[5]; 6 7 public: 8 int& operator[](int i) { 9 if(i < 0 i >= 5) { 10 cout << "Index Out of Bounds\n"; 11 exit(0); 12 } 13 return a[i]; 14 } 15 }; 16 17 int main() { 18 Array arr; 19 20 arr[0] = 10; 21 cout << arr[0] << endl; 22 23 return 0; 24 }</pre>	<pre>10 === Code Execution Successful ===</pre>

9.

Overload `()` to input arbitrary number of input arguments for an object.

main.cpp	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 class Demo { 5 public: 6 void operator()(int a, int b, int c) { 7 cout << "Values: " << a << " " << b << " " << c << endl; 8 } 9 }; 10 11 int main() { 12 Demo d; 13 d(1,2,3); 14 15 return 0; 16 }</pre>	<pre>Values: 1 2 3 === Code Execution Successful ===</pre>

10.

Write a program in C++ to overload:

- Input operator >>
- Output operator <<

main.cpp	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 class Student { 5 int roll; 6 7 public: 8 friend istream& operator>>(istream &in, Student &s 9) { 10 cout << "Enter Roll: "; 11 in >> s.roll; 12 return in; 13 } 14 friend ostream& operator<<(ostream &out, Student 15 &s) { 16 out << "Roll No: " << s.roll; 17 return out; 18 }; 19 20 int main() { 21 Student s; 22 23 cin >> s; 24 cout << s; 25 26 return 0; 27 }</pre>	<pre>Enter Roll: 1024150212 Roll No: 1024150212 === Code Execution Successful ===</pre>

11.

Write a program to convert basic data type (float) to user-defined data type (object) using constructor.

main.cpp	Output
<pre>1 #include <iostream> 2 using namespace std; 3 4 class Test { 5 float x; 6 7 public: 8 Test(float a) { 9 x = a; 10 } 11 12 void show() { 13 cout << x << endl; 14 } 15 }; 16 17 int main() { 18 float a = 5.5; 19 20 Test t = a; 21 t.show(); 22 23 return 0; 24 }</pre>	<pre>5.5 === Code Execution Successful ===</pre>

12.

Write a program to convert user-defined data type (UDT) to basic data type (float) using conversion operator.

<pre>1 #include <iostream> 2 using namespace std; 3 4 class Test { 5 float x; 6 public: 7 Test(float a) { 8 x = a; 9 } 10 operator float() { 11 return x; 12 }; 13 int main() { 14 Test t(5.5); 15 float a = t; 16 cout << a << endl; 17 return 0; 18 }</pre>	<pre>5.5 === Code Execution Successful ===</pre>
--	---

13.

Convert one UDT to another UDT (example: Polar to Cartesian).

Polar p(10,5);

Cartesian c = p;

c.show();

main.cpp

Share

Run

Output

Clear

```
1 #include <iostream>
2 #include <cmath>
3 using namespace std;
4
5 class Polar {
6     float r, theta;
7
8 public:
9     Polar(float a, float b) {
10         r = a;
11         theta = b;
12     }
13
14     float getR() { return r; }
15     float getTheta() { return theta; }
16 };
17
18 class Cartesian {
19     float x, y;
20
21 public:
22     Cartesian(Polar p) {
23         x = p.getR() * cos(p.getTheta());
24         y = p.getR() * sin(p.getTheta());
25     }
26
27     void show() {
28         cout << "x = " << x << ", y = " << y << endl;
29     }
30 };
31
32 int main() {
33     Polar p(10,5);
34     Cartesian c = p;
35
36     c.show();
37
38     return 0;
```

x = 2.83662, y = -9.58924

=== Code Execution Successful ===